

Mathematical Foundations of Wideband Beamforming

An Engineering Guide for Acoustic Localization

1 Introduction

Beamforming is the process of using a spatial array of sensors (microphones) to “listen” in a specific direction while suppressing noise from others. In our case, we use a **Uniform Linear Array (ULA)**. Because sound (speech, noise) is not a single frequency but a collection of many frequencies, we must use **Wideband Beamforming**.

2 The Physics of Delay

Imagine two microphones separated by a distance d . A sound wave coming from an angle θ (relative to the perpendicular “broadside”) must travel a slightly longer distance to reach one microphone compared to the other.

$$\Delta x = d \sin(\theta)$$

The time it takes for sound to travel this extra distance is the **Time of Arrival Difference (TDOA)**, denoted by τ :

$$\tau(\theta) = \frac{d \sin(\theta)}{c}$$

where $c \approx 343$ m/s is the speed of sound. If we have N microphones at positions $x_n = n \cdot d$, the delay for the n -th microphone relative to the first one is:

$$\tau_n(\theta) = \frac{n \cdot d \sin(\theta)}{c}$$

Code Mapping (Section 5 of Notebook):

```
# Constants defined in notebook:
c = 343
f = 4000
d = (c/f) / 2 # Half-wavelength spacing

# Geometry Calculation:
mic = np.arange(1, 16) - 8 # Microphone indices [-7, -6, ..., 7]
steering_angles = np.deg2rad(np.linspace(0, 180, 100))

# The Core Physics:
# tau_n = d * cos(theta) * mic / c (Note: Notebook uses cos for 1D axial)
delay_term = d * np.cos(steering_angles) * mic
```

3 From Time to Frequency (The FFT)

Processing signals in the time domain is difficult due to sample rate resolution. We shift to the **Frequency Domain**.

3.1 Understanding `numpy.fft`

When you call `np.fft.rfft(signal)`, NumPy performs a Real Fast Fourier Transform.

1. **Input:** A block of N audio samples.
2. **Output:** A complex array.

- **Magnitude** $|X(f)|$: Energy at frequency f .
- **Phase** $\angle X(f)$: Timing at frequency f .

3.2 Corollary: Phase as Timing

Claim: The phase φ of the Fourier component $X(f)$ is directly proportional to the time delay τ .

Proof: Consider a tone $s(t) = \cos(2\pi ft)$. In the complex plane: $e^{j2\pi ft}$. Delayed signal: $s(t - \tau) = \cos(2\pi f(t - \tau))$. Argument: $2\pi ft - 2\pi f\tau$. The offset $\varphi = -2\pi f\tau$. Therefore: $\tau = -\frac{\varphi}{2\pi f}$. Thus, shifting phase is exactly shifting time.

Key Insight: In our code, we don't "wait" for sound to arrive. We rotate its complex phase.

4 The Steering Vector and Matrix

To "steer" the array, we define a weight vector w that cancels the physical delays. For a frequency f and angle θ :

$$a(f, \theta) = \begin{pmatrix} e^{-j2\pi f\tau_0(\theta)} \\ \dots \\ e^{-j2\pi f\tau_{N-1}(\theta)} \end{pmatrix}$$

Notebook Snippet (Wideband Setup):

```
# 'k' is the wave number array [2*pi*f/c] for 100 frequencies
k = 2 * np.pi / c * np.linspace(1000, 4001, 100)

# Steering Matrix W: (Frequency, Mic, Angle)
W = np.exp(-1j * d * k[:, None, None] *
           mic[None, :, None] *
           np.cos(steering_angles[None, None, :]))
```

Built-in Explanation:

- `np.exp(-1j * ...)`: Calculates the Euler identity $\cos(x) - j\sin(x)$.
- `[:, None, None]`: This is **broadcasting**. It turns a 1D array into 3D so we can multiply all frequencies, microphones, and angles in one line.

5 SRP-PHAT: The Gold Standard

Simple beamforming fails with echoes. We use **SRP-PHAT**.

5.1 The Phase Transform (PHAT)

We normalize frequencies to magnitude 1:

$$\hat{X}_n(f) = \frac{X_n(f)}{|X_n(f)| + \varepsilon}$$

Function Explanation: `np.abs(X)` calculates magnitude. Dividing by it "whitens" the signal, making quiet and loud frequencies equally important for timing.

5.2 Wideband Summation (The Einstein Sum)

Notebook Snippet (The "Magic" Line):

```
# Y: Beamformed output in Frequency domain
# 'fms': Steering Matrix (Frequency, Mic, Angle)
# 'fm': Signal (Frequency, Mic)
# 'fs': Result (Frequency, Angle)
Y = np.einsum('fms, fm -> fs', W.conj(), signal_aligned)
```

```
# P: Total Power (Wideband Summation)
P = np.mean(np.abs(Y)**2, axis=0)
```

6 Glossary of Functions

- `np.fft.fft`: Calculates the complex spectrum.
- `np.fft.fftshift`: Shifts the zero-frequency component to the center of the spectrum.
- `np.mean(..., axis=0)`: Averages across frequencies to get the wideband result.
- `W.conj()`: Takes the complex conjugate (flips the sign of the imaginary part). This is required to “undo” the delay during steering.